

A-Smart-Safe-and-Automated-Gate

1. Project Introduction

This project is a **smart, automated gate system** that simulates a real-world entry system. It automatically opens for an approaching object and closes after a set interval. It includes an ultrasonic sensor for detection, a DC motor for the gate's movement, and LEDs and a buzzer for visual and audible alerts. This project is a great demonstration of how sensors, motor control, and conditional logic can be combined to create a practical, automated system.

- **Automatic Operation:** The gate uses an ultrasonic sensor to detect objects.
 - **Timed Intervals:** It remains open for a specific duration before automatically closing.
 - **Visual & Audible Alerts:** A red LED, a green LED, and a buzzer provide clear feedback on the gate's status.
-

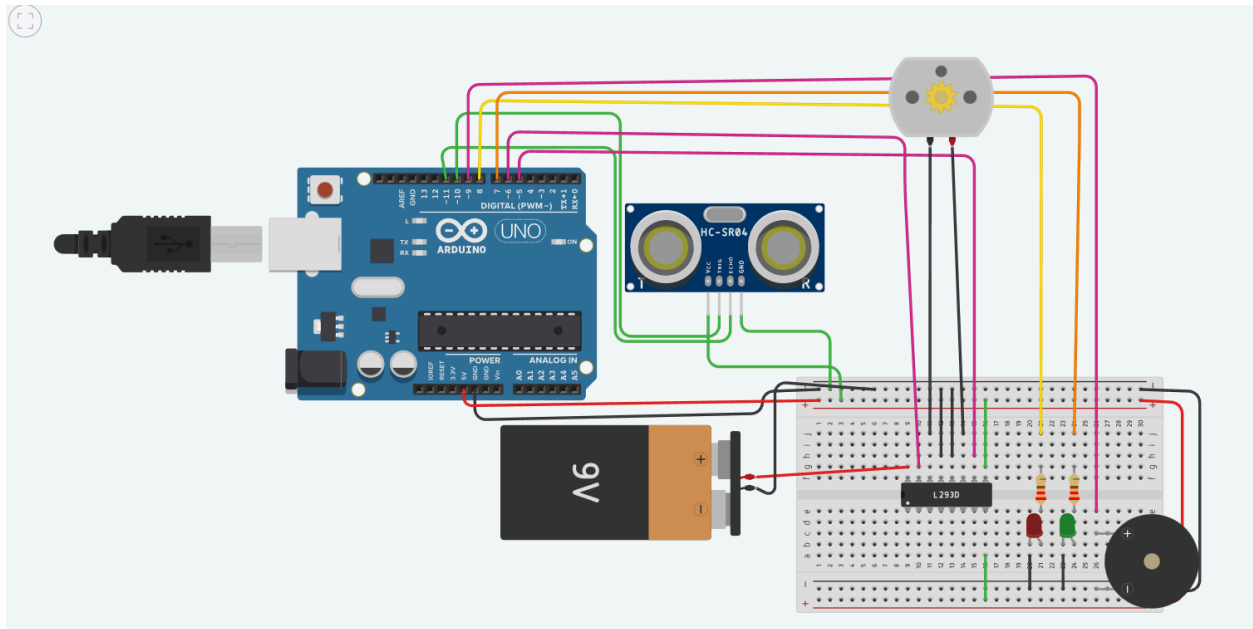
2. Components Required

This is a list of all the hardware you need to build the project.

Component	Quantity	Purpose
Arduino Uno	1	The "brain" of the project that controls everything.
DC Motor	1	Simulates the gate motor.
H-Bridge Motor Driver	1	A circuit that controls the motor's direction and speed.
Ultrasonic Sensor (HC-SR04)	1	Detects objects by measuring distance with sound waves.
9V Battery	1	Provides high-current power for the DC motor.
Red & Green LEDs	1 of each	Acts as a traffic light to show the gate's status.
Piezo Buzzer	1	Provides audible warnings when the gate is in motion.
220Ω Resistors	2	Protects the LEDs from high current.

Breadboard	1	A board for prototyping circuits without soldering.
Jumper Wires	Assorted	Used to connect all the components.

<https://docs.google.com/spreadsheets/d/1Ph2H8ubv9v-uW3EP70ydssEju4r9XpxOwR8GiKOH448/edit?usp=sharing>



4. Arduino Code

The provided code implements a **state machine**, which is a professional way to manage complex logic. The system moves between five states: **CLOSED**, **OPENING**, **OPEN**, **CLOSING**, and **STOPPED**.

```
// Define all pin numbers
const int motorPin1 = 5;
const int motorPin2 = 6;
const int greenLEDPin = 7;
const int redLEDPin = 8;
const int buzzerPin = 9;
const int trigPin = 10;
const int echoPin = 11;
```

```
// Define states for our gate's "brain"
enum GateState { CLOSED, OPENING, OPEN, CLOSING, STOPPED };
GateState currentState = CLOSED; // Start in the CLOSED state
```

```
// Variables for timing (using millis() for non-blocking code)
unsigned long openGateStartTime = 0;
unsigned long closeGateStartTime = 0;
const unsigned long openDuration = 4000; // Time in milliseconds for the gate to fully open
const unsigned long closeDuration = 4000; // Time in milliseconds for the gate to fully close
const unsigned long gateOpenWaitTime = 10000; // Time the gate stays open before closing
```

```
// Global variables for sensor readings
long duration, distance;
```

```
// Helper function to stop the motor
void stopMotor() {
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, LOW);
}
```

```
void setup() {
    // Set up all your pins
    pinMode(motorPin1, OUTPUT);
    pinMode(motorPin2, OUTPUT);
    pinMode(greenLEDPin, OUTPUT);
    pinMode(redLEDPin, OUTPUT);
    pinMode(buzzerPin, OUTPUT);
    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
```

```
    Serial.begin(9600); // For debugging
```

```
    // Initial state setup
    stopMotor();
    digitalWrite(redLEDPin, HIGH); // Red LED is on by default to show "closed"
    digitalWrite(greenLEDPin, LOW);
    noTone(buzzerPin);
}
```

```
void loop() {
    // Read distance from ultrasonic sensor
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
    duration = pulseIn(echoPin, HIGH);
```

```
distance = duration * 0.034 / 2;
```

```
// The state machine logic
```

```
switch (currentState) {
```

```
  case CLOSED:
```

```
    // Red LED is on and no buzzer
```

```
    digitalWrite(redLEDPin, HIGH);
```

```
    digitalWrite(greenLEDPin, LOW);
```

```
    noTone(buzzerPin);
```

```
    // Check for a vehicle to open the gate
```

```
    if (distance < 50 && distance > 0) {
```

```
      currentState = OPENING;
```

```
      openGateStartTime = millis(); // Start the opening timer
```

```
    }
```

```
    break;
```

```
  case OPENING:
```

```
    // Start the motor to open
```

```
    digitalWrite(motorPin1, HIGH);
```

```
    digitalWrite(motorPin2, LOW);
```

```
    digitalWrite(redLEDPin, LOW);
```

```
    digitalWrite(greenLEDPin, HIGH);
```

```
    tone(buzzerPin, 1000);
```

```
    // Check if the motor has run for the specified openDuration
```

```
    if (millis() - openGateStartTime >= openDuration) {
```

```
      stopMotor();
```

```
      currentState = OPEN;
```

```
      closeGateStartTime = millis(); // Start the timer for how long the gate stays open
```

```
    }
```

```
    break;
```

```
  case OPEN:
```

```
    digitalWrite(redLEDPin, LOW);
```

```
    digitalWrite(greenLEDPin, HIGH);
```

```
    noTone(buzzerPin);
```

```
    // Check if the timer to close has expired AND a vehicle is no longer detected
```

```
    if (millis() - closeGateStartTime >= gateOpenWaitTime && distance > 50) {
```

```
      currentState = CLOSING;
```

```
      closeGateStartTime = millis(); // Start the closing timer
```

```
    }
```

```
    break;
```

```

case CLOSING:
    // Start the motor to close
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, HIGH);
    digitalWrite(redLEDPin, HIGH);
    digitalWrite(greenLEDPin, LOW);
    tone(buzzerPin, 1000);

    // CRUCIAL SAFETY CHECK: If a person or object is detected
    if (distance < 50 && distance > 0) {
        currentState = STOPPED;
    }

    // Check if the motor has run for the specified closeDuration
    if (millis() - closeGateStartTime >= closeDuration) {
        stopMotor();
        currentState = CLOSED;
    }
    break;

case STOPPED:
    stopMotor();
    noTone(buzzerPin);
    // Flash the red LED to indicate an error/stop
    digitalWrite(redLEDPin, !digitalRead(redLEDPin));
    delay(100);

    // Resume closing if no obstacle is detected
    if (distance > 50) {
        currentState = CLOSING;
        closeGateStartTime = millis(); // Resume the closing timer from this point
    }
    break;
}
}

```

Briefly explaining code here:

https://docs.google.com/document/d/1_NGFSeqMKJNFhN2Rd58s8n862FpMk71IObBtHakCiBs/edit?usp=sharing

5. How to Use the System

1. **Power On:** Plug the Arduino into your computer. The red LED will light up, indicating the gate is in the **CLOSED** state.
2. **Trigger Open:** Place your hand or an object in front of the ultrasonic sensor. The buzzer will sound, the green LED will turn on, and the motor will spin to open the gate.
3. **Automatic Close:** The motor will stop after its set run time. The gate will wait for 10 seconds, then the buzzer will sound again, and the motor will spin in the reverse direction to close the gate.
4. **Safety Stop:** While the gate is closing, place your hand in its path. The motor will immediately stop, and the red LED will flash, demonstrating the safety feature. The gate will resume closing once the obstacle is removed.

6. Challenges Faced and Solutions.

The Code and Compiler Errors

Early in the project, we encountered several compiler errors. The most common issues were:

- **Variable Scope:** Variables like `stopMotor()` and `closeGateStartTime` were not recognized in certain parts of the code because they were defined in the wrong place. The solution was to define all functions and global variables at the top of the sketch before the `setup()` function.
- **Variable Re-declaration:** We had unintentionally declared the same variable twice, which the compiler flagged as an error.

Hardware Integration and Motor Failure

The biggest challenge was debugging the physical circuit. After confirming the code was correct and the program was entering the **OPENING** state, the motor still would not spin. This pointed to a hardware problem. Through a systematic debugging process, we discovered several critical wiring issues.

- **Lack of a Separate Power Supply:** Initially, the motor was not connected to an external power source. A DC motor requires far more current than the Arduino's 5V pin can provide, making an external power source essential.
- **Missing Common Ground:** The most common and subtle issue was the lack of a common ground connection. For the H-bridge to understand the control signals from the Arduino, both components must share the same electrical reference point, which is achieved by connecting their **GND pins together**.

- **H-Bridge Enable Pin:** We discovered that the motor would not spin unless the H-bridge's **enable pin (ENA)** was set to **HIGH**. Connecting this pin directly to the Arduino's 5V pin solved this issue and allowed the control signals to reach the motor.